App Dev 1 Project Report

Student details

Name: Rohan Uday Puntambekar

Roll Number: 21f3002298

Email: 21f3002298@ds.study.iitm.ac.in

I am a BS Degree student at IIT Madras with an interest in artificial intelligence and machine learning research. Apart from pursuing this diploma, I am currently a Master's student in Artificial Intelligence at Northeastern University, Portland, Maine, in the United States. This project was completed as part of the Modern Application Development I course in the January 2026 term.

Project details

Project Title: Placement Portal Application

Problem Statement: Institutes need a structured system to manage campus recruitment activities involving companies, students, and placement drives. The objective of this project is to build a web application that allows an Admin (institute placement cell), Companies, and Students to interact with the system based on their roles.

Approach: I began by analyzing and identifying the key entities: User, Company, Student, PlacementDrive, and Application. I designed a unified User table for authentication across all three roles, with separate Company and Student tables holding role-specific profile data linked via foreign keys. The development followed the milestones provided to us by the IITM App Dev course team. I first set up the database models and relationships using SQLAlchemy, then implemented authentication with Flask-Login. Role-based access control was built by checking the user's role attribute before granting access to any route. After authentication was working, I built the admin dashboard with company/student management features, followed by the company dashboard with drive creation and applicant management, and finally the student dashboard with drive browsing and application tracking.

AI/LLM declaration

I used Claude (Anthropic) to assist in developing this project. The AI was used to debug and refine the Flask application structure, SQLAlchemy model definitions and relationships, and resolve errors such as circular imports between modules. I used AI to generate a Jinja2 template skeleton, then filled in the details of each page myself. I also consulted AI suggestions for implementing Flask-Login authentication and the *secure_filename* pattern for document upload. All business logic decisions, database schema, runtime testing, end-to-end workflow verification, integration decisions, and bug fixes during testing were completely done by me. The estimated extent of AI/LLM usage is around 15-20%.

Technologies and frameworks used

Flask serves as the core backend framework. It handles URL routing, request processing, and response generation.

SQLAlchemy (Flask-SQLAlchemy) is used as the Object Relational Mapper. It allows defining database tables as Python classes and querying them without writing raw SQL. This made it straightforward to define

relationships between entities such as Company-to-PlacementDrive (one-to-many) and Student-to-Application (one-to-many).

Flask-Login manages user sessions and authentication. It provides the *login_user*, *logout_user*, and *login_required* utilities that handle session cookies and protect routes from unauthenticated access.

Jinja2 is Flask's built-in template engine used for rendering HTML pages on the server side. Each role has its own set of templates that extend a shared base template containing the navbar and flash message display.

Bootstrap 5 (CDN) provides CSS styling. All pages use Bootstrap classes for layout, tables, cards, forms, and alerts. No custom CSS framework was used.

SQLite is the database. It stores all application data in a single file (*placement.db*) and is created programmatically through SQLAlchemy's *db.create_all()* method.

Werkzeug is used for secure password hashing (*generate_password_hash / check_password_hash*) and safe file uploads (*secure_filename*).

Database schema and ER diagram

The database consists of five tables with the following relationships:

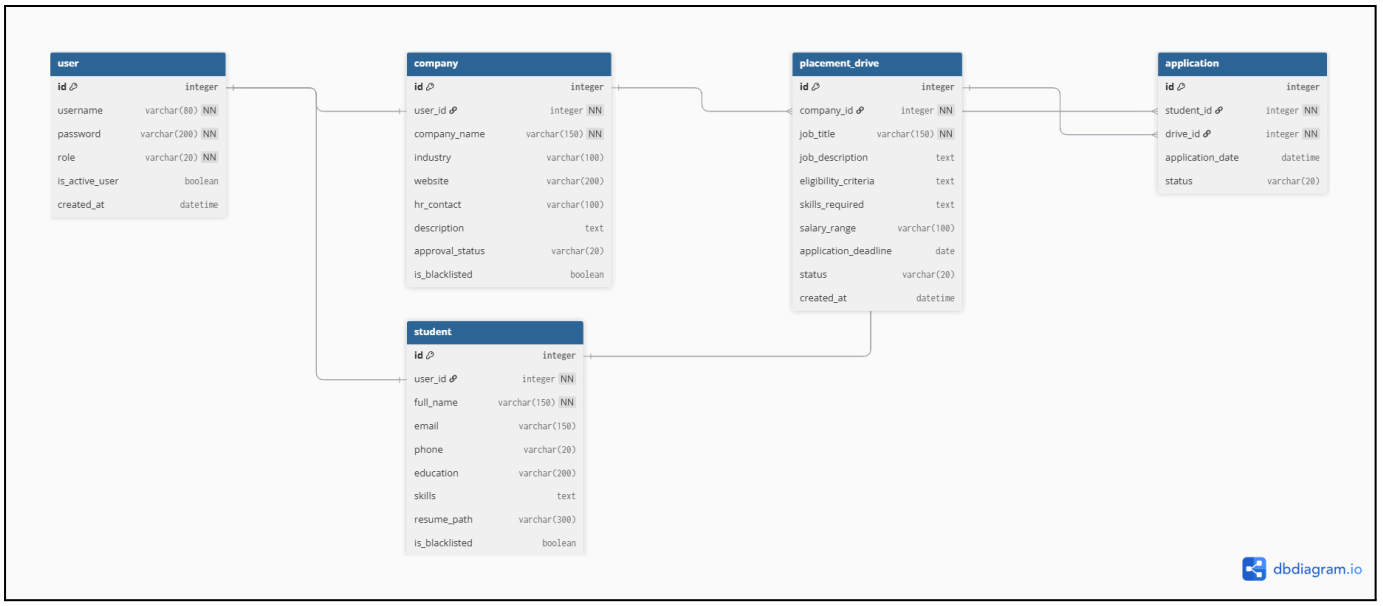
User is the central authentication table. It stores login credentials and a role field indicating whether the user is an admin, a company, or a student. Each User has a one-to-one relationship with either a Company or a Student record.

Company stores the company profile (name, industry, website, HR contact, description). It has an *approval_status* field, which the admin controls, and an *is_blacklisted* flag. A company can create multiple PlacementDrives (one-to-many).

Student stores the student profile (name, email, phone, education, skills, resume path). It also has an *is_blacklisted* flag. A student can submit multiple Applications (one-to-many), but not for the same PlacementDrive.

PlacementDrive represents a recruitment drive posted by a company. It includes the job title, description, eligibility criteria, required skills, salary range, deadline, and approval status. Each drive can receive multiple Applications (one-to-many).

Application links a student to a placement drive. It records the application date and status (applied, shortlisted, selected, rejected). A unique constraint on (student_id, drive_id) prevents duplicate applications.



API resource endpoints

This project uses server-rendered Jinja2 templates rather than a dedicated REST API. All interactions happen through standard Flask routes that return HTML responses. No JSON-based API endpoints were implemented.

Architecture and features

Project structure: The application follows a single-module structure with *app.py* as the main entry point containing all routes, *models.py* defining the database schema, *config.py* holding configuration, and *extensions.py* initializing Flask extensions. Templates are organized in role-specific subdirectories under *templates/*.

Implemented core features:

Authentication supports login for all three roles through a shared login page. Registration is available only to companies and students. The admin account is preseeded in the database during the first run. Companies must be approved by the admin before they can log in. Blacklisted users are prevented from logging in.

The admin dashboard displays summary statistics (total companies, students, drives, applications, and pending counts). Admin can approve or reject company registrations and placement drives, search companies by name or industry, search students by name, email, or phone, and blacklist or unblacklist any company or student.

The company dashboard displays company details and a list of created drives, along with the number of applications for each drive. Companies can create new placement drives (only if approved), close or reopen drives, view applications per drive, and update application status to shortlisted, selected, or rejected. Student profiles and resumes are accessible from the applications view.

The student dashboard shows available drive count, application count, and notifications for status changes. Students can browse and search approved drives, apply to drives (with duplicate prevention enforced at both application and database levels), view their application history with status, and edit their profile, including resume upload.

Video presentation

Drive Link: https://drive.google.com/file/d/1T9z9aaly0swPkvbvQvjwZ2BsPrvaMCfR/view?usp=sharing